Data skew 2 — skew dataset simulation

annotated run of `spark/1_data_skew/2_skew_dataset_simulation.ipynb` · every output below is from a live run: Spark 4.3.0, local[4], 39,790,092 ⋈ 5,000 rows · video: youtu.be/9Ss-_y7njKE (playlist item 8)

Imports & Configuration

```
from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = 'all'

import warnings
warnings.filterwarnings('ignore')
warnings.simplefilter('ignore')
import time
```

```
from pyspark.sql.types import IntegerType
import pyspark.sql.functions as F
from pyspark.sql import SparkSession
```

```
spark = SparkSession.builder.master("local[4]").getOrCreate()
sc = spark.sparkContext
sc.setLogLevel("ERROR")
```

```
Spark 4.3.0  |  master = local[4]  |  UI at http://localhost:4041
```

```
# spark.conf.set("spark.sql.shuffle.partitions", "3")
spark.conf.set("spark.sql.adaptive.enabled", "false")
```

✍️ Why the chapter switches AQE off

- **`spark.sql.adaptive.enabled=false`** turns off Adaptive Query Execution.
- AQE re-plans at runtime from real shuffle statistics: it coalesces tiny partitions and can split a skewed one. Left on, it would hide the very thing this chapter wants you to see.
- Spark 3.2+ and 4.x ship AQE **on**, so this line is a deliberate downgrade to classic behaviour.

💡 AQE off to **observe** skew, AQE on to **survive** it.

Simulating Uniform Dataset

```
df_uniform = spark.range(1000000)
df_uniform.show(3, False)
```

```
+---+
|id |
+---+
|0  |
|1  |
|2  |
+---+
```

```
(
    df_uniform
    .withColumn("partition", F.spark_partition_id())
    .groupBy("partition")
    .count()
    .orderBy("partition")
    .show()
)
```

```
+---------+------+
|partition|count |
+---------+------+
|0        |250000 ①|
|1        |250000|
|2        |250000|
|3        |250000|
+---------+------+
```

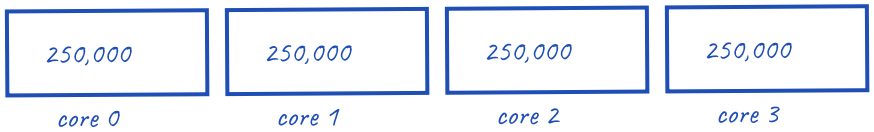
✍️ Definition: partition ⟶ task

- A **partition** is one slice of the DataFrame. A stage runs exactly **one task per partition.**
- `F.spark_partition_id()` stamps every row with the partition it sits in right now - the cheapest skew probe there is.
- 4 partitions because `master=local[4]`: default parallelism = number of cores.

① `spark.range(1_000_000)` ⟶ 4 × 250,000. `range` splits by row number, not by a key, so it cannot skew.

↗ match the circles in the output above

uniform: spark.range(1_000_000) on local[4]



all four finish together

💡 Even partitions ⇒ every core finishes at the same time. This is the baseline the next cell breaks on purpose.

Skewed Dataset

```
df0 = spark.range(0, 1000000).repartition(1)
df1 = spark.range(0, 10).repartition(1)
df2 = spark.range(0, 10).repartition(1)
df_skew = df0.union(df1).union(df2)
df_skew.show(3, False)
```

```
+---+
|id |
+---+
|0  |
|1  |
|2  |
+---+
only showing top 3 rows
```

```
(
    df_skew
    .withColumn("partition", F.spark_partition_id())
    .groupBy("partition")
    .count()
    .orderBy("partition")
    .show()
)
```

```
numPartitions = 1
+---------+-------+
|partition|count  |
+---------+-------+
|0        |1000020| ①|
+---------+-------+
```

✍️ Definition: data skew - and a Spark 4 gotcha

- **Data skew** = rows spread unevenly; one partition (one key) holds most of them.
- One task per partition ⇒ the fat partition becomes one long task, and a stage is finished only when its **last** task is.
- Symptoms: cores idle, wall-clock set by a single task, spill to disk, OOM on one executor.

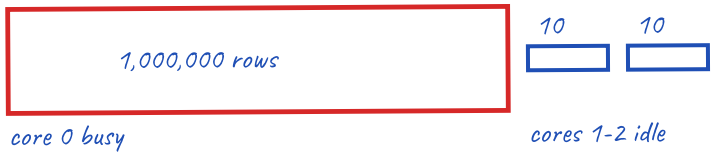
① This run prints **one** partition of 1,000,020 - not the 1,000,000 / 10 / 10 of the video.

↗ match the circles in the output above

⚠ **Gotcha (Spark 4.3):** all three unioned frames come out of `repartition(1)`, so they share the same *SinglePartition* partitioner. `SparkContext.union` then builds a **PartitionerAwareUnionRDD** and merges partition 0 of each child into one. DataFrame `union` no longer hands you three partitions.

Reproduce the video's picture by unioning at RDD level:

`spark.createDataFrame(df0.rdd.union(df1.rdd).union(df2.rdd), df0.schema)` ⟶ 3 partitions: 1,000,000 / 10 / 10



💡 The lesson is unchanged, the demo is not. Never trust a notebook's partition layout - print `spark_partition_id()` yourself.

Join Skews

```
transactions_file = "../../data/data_skew/transactions.parquet"
customer_file = "../../data/data_skew/customers.parquet"

df_transactions = spark.read.parquet(transactions_file)
df_customers = spark.read.parquet(customer_file)
```

> ✍️ **Windows gotcha while reading the parquet**
>
> `spark.read.parquet("…/transactions.parquet")` on Windows without `winutils.exe` + `hadoop.dll` (HADOOP_HOME unset) dies with `UnsatisfiedLinkError: NativeIO$Windows.access0`.
>
> - Only the **directory listing** needs that native call (`FileUtil.canRead`); reading the files themselves does not.
> - Workaround used for this run: hand Spark the part-files directly - `spark.read.parquet(*glob.glob(path + "/part-*.parquet"))`.
>
> 💡 163 part-files of transactions (871 MB) + 1 of customers (161 KB) = 39,790,092 and 5,000 rows.

```
df_transactions.printSchema()
df_transactions.show(5, False)
```

```
root
 |-- cust_id: string (nullable = true)
 |-- start_date: string (nullable = true)
 |-- end_date: string (nullable = true)
 |-- txn_id: string (nullable = true)
 |-- date: string (nullable = true)
 |-- year: string (nullable = true)
 |-- month: string (nullable = true)
 |-- day: string (nullable = true)
 |-- expense_type: string (nullable = true)
 |-- amt: string (nullable = true)
 |-- city: string (nullable = true)

+----------+----------+----------+--------------+----------+----+-----+---+-------------+------+-----------+
|cust_id   |start_date|end_date  |txn_id        |date      |year|month|day|expense_type |amt   |city       |
+----------+----------+----------+--------------+----------+----+-----+---+-------------+------+-----------+
|C0YDPQWPBJ|2010-07-01|2018-12-01|TZ5SMKZY9S03OQJ|2018-10-07|2018|10   |7  |Entertainment|10.42 |boston     |
|C0YDPQWPBJ|2010-07-01|2018-12-01|TYIAPPNU066CJ5R|2016-03-27|2016|3    |27 |Motor/Travel |44.34 |portland   |
|C0YDPQWPBJ|2010-07-01|2018-12-01|TETSXIK4BLXHJ6W|2011-04-11|2011|4    |11 |Entertainment|3.18  |chicago    |
|C0YDPQWPBJ|2010-07-01|2018-12-01|TQKL1QFJY3EM8LO|2018-02-22|2018|2    |22 |Groceries    |268.97|los_angeles|
|C0YDPQWPBJ|2010-07-01|2018-12-01|TYL6DFP09PPXMVB|2010-10-16|2010|10   |16 |Entertainment|2.66  |chicago    |
+----------+----------+----------+--------------+----------+----+-----+---+-------------+------+-----------+
```

```
df_customers.printSchema()
df_customers.show(5, False)
```

```
root
 |-- cust_id: string (nullable = true)
 |-- name: string (nullable = true)
 |-- age: string (nullable = true)
 |-- gender: string (nullable = true)
 |-- birthday: string (nullable = true)
 |-- zip: string (nullable = true)
 |-- city: string (nullable = true)
```

```
+----------+-------------+---+------+----------+-----+-----------+
|cust_id   |name         |age|gender|birthday  |zip  |city       |
+----------+-------------+---+------+----------+-----+-----------+
|C007YEYTX9|Aaron Abbott |34 |Female|7/13/1991 |97823|boston     |
|C00B971T1J|Aaron Austin |37 |Female|12/16/2004|30332|chicago    |
|C00WRSJF1Q|Aaron Barnes |29 |Female|3/11/1977 |23451|denver     |
|C01AZWQMF3|Aaron Barrett|31 |Male  |7/9/1998  |46613|los_angeles|
|C01BKUFRHA|Aaron Becker |54 |Male  |11/24/1979|40284|san_diego  |
+----------+-------------+---+------+----------+-----+-----------+
```

```python
(
    df_transactions
    .groupBy("cust_id")
    .agg(F.countDistinct("txn_id").alias("ct"))
    .orderBy(F.desc("ct"))
    .show(5, False)
)
```

```
+----------+--------+
|cust_id   |ct      |
+----------+--------+
|C0YDPQWPBJ|17539732| ①
|C3KUDEN3KO|7999    | ②
|CBW3FMEAU7|7999    |
|C89FCEGPJP|7999    |
|CHNFNR89ZV|7998    |
+----------+--------+
```

✍️ Find the skewed key before fixing anything

① `C0YDPQWPBJ` ⟶ 17,539,732 rows = **44 %** of all 39,790,092 rows, in **one** key.

② next key ⟶ 7,999 rows. Key-level ratio = **2,193 ×**.

match the circles in the output above

- Recipe: `df.groupBy(key).count().orderBy(F.desc("count")).show(5)`.
- The notebook's `countDistinct("txn_id")` took **33.9 s** against **1.5 s** for a plain count: it shuffles on (cust_id, txn_id) first, so two shuffles instead of one. For hunting skew a plain count is enough.

💡 Skew ratio = max / median. >5× is a smell, >50× is a fire. Here it is 160× on rows read per task.

```python
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", -1)
```

✍️ Why broadcast is killed on purpose

- customers = 5,000 rows / 161 KB - far below the 10 MB auto-broadcast threshold, so Spark would broadcast it, there would be **no shuffle**, and no skew to look at.
- `autoBroadcastJoinThreshold = -1` forces a **SortMergeJoin**: both sides get hash-partitioned on cust_id, so all 17.5 M rows of one customer land in **one** of the 200 partitions.

⚠ **Gotcha:** this is a teaching switch, not a tuning tip. For this exact pair of tables broadcast is the fix - measured on this machine: **1.05 s** with broadcast vs **8.26 s** without.

```
df_txn_details = (
    df_transactions.join(
        df_customers,
        on="cust_id",
        how="inner"
    )
)
```

```
start_time = time.time()
df_txn_details.count()
print(f"time taken: {time.time() - start_time}")
```

```
39790092 ①
time taken: 7.82 s ②
```

✍️ Code ⟶ output: what the 7.82 s hides

① 39,790,092 rows joined ② 7.82 s wall clock on 4 cores.

↗ match the circles in the output above

Spark UI, **stage 27** (the join stage) from this very run:

metric	median task	max task
duration	33 ms	4 s (134 ×)
shuffle read records	110,390	17,684,814 (160 ×)
spill (memory / disk)	0 B	416 MiB / 2.8 MiB

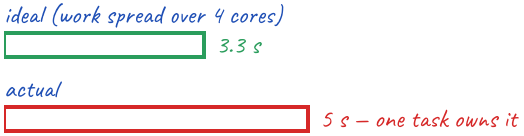
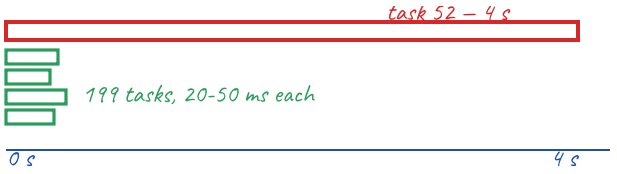
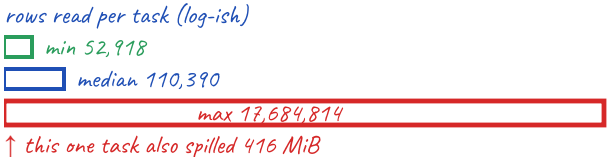
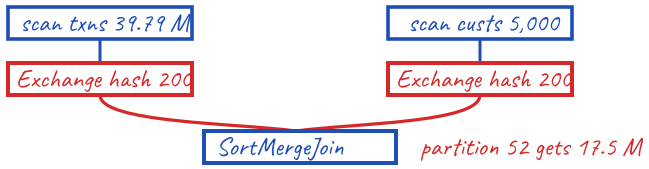
199 tasks were done inside the first ~0.6 s; the stage then sat waiting for **task 52**.

💡 A stage costs what its slowest task costs. Average task time is a lie once data is skewed.

```
# spark.stop()
```

Where the Spark UI shows skew — five screens, one run

all numbers below are from this machine: 39,790,092 transactions ⋈ 5,000 customers, sort-merge join, AQE off, `local[4]`, job 13 / stage 27

Screen	Rule to apply	What this run showed	Picture
Jobs / Stages `/stages/`	Compare *Duration* with *Total time across all tasks ÷ cores*. If duration is much bigger, one task is holding the stage.	stage 27: 200 tasks, total task time **13 s**, 4 cores ⇒ ideal ≈ 3.3 s. Actual duration **5 s**.	
Event timeline stage page	One green bar that runs on after every other bar has stopped = the skewed partition.	199 tasks finish inside the first **~0.6 s**; task **52** keeps running for **4 s**.	
Summary metrics stage page	Read the *Max* column against *Median*. Ratio > 5× = skew; also look at Spill.	duration 33 ms ⟶ **4 s** (134×) read 110,390 ⟶ **17,684,814** recs (160×) spill **416 MiB** mem / 2.8 MiB disk	
Task table stage page	Filter the table for `MiB` (or sort by Shuffle Read). A skewed stage has exactly one row in another unit.	index **52** / task 1300: 4 s, 3.5 MiB / 17,684,814 recs, spill 416 MiB. Its neighbours: ~24 ms, ~14 KiB.	
SQL / DataFrame query plan	Find the `Exchange hashpartitioning(key, 200)`. That line is where skew is decided — same key, same partition, always.	left edge 39,790,092 rows, right edge 5,000; both hashed on `cust_id` into 200, then `SortMergeJoin`.	

💡 *Order to check: Stages list (is one stage long?) ⟶ Summary metrics (Max vs Median) ⟶ Event timeline (one long bar?) ⟶ Task table (which partition?) ⟶ SQL plan (which Exchange made it?).*

Data skew — chapter 2: skew dataset simulation

Typed notes to go with the notebook pages. Everything measured below is from one local run on 2026-11-03: Spark 4.3.0, `local[4]`, transactions 39,790,092 rows (871 MB parquet) ⋈ customers 5,000 rows (161 KB).

1 ★ What this chapter does — and what it does not

Chapter 2 is a DIAGNOSIS chapter. It manufactures skew and teaches you to see it. It contains no fix.
Nothing in `2_skew_dataset_simulation.ipynb` makes the skewed join faster. Every line either creates skew or measures it.

Notebook cell	What it proves	Is it a fix?
`spark.sql.adaptive.enabled = false`	AQE would hide skew, so it is turned off	**No — it removes the fix Spark gives you for free**
uniform `spark.range` + `spark_partition_id()`	what "even" looks like: 4 × 250,000	**No — baseline only**
union of three `repartition(1)` frames	what "skewed partitions" look like	**No — it *creates* skew on purpose**
`groupBy(cust_id).countDistinct(txn_id)`	finds the hot key: one customer = 44 % of the table	**No — it is the diagnosis step**
`autoBroadcastJoinThreshold = -1`	forces a sort-merge join so the shuffle is visible	**No — it *disables* the fix that would have worked here**
`df_txn_details.count()` + Spark UI	one task reads 160× the median and spills 416 MiB	**No — it is the measurement**

Where the fixes live: AQE and broadcast are chapter 3 (`3_solving_data_skew_aqe_broadcast.ipynb`), salting is chapter 4 (`4_salting.ipynb`). Section 4 below reproduces all of them against this same dataset so the numbers are comparable.

> **One-liner:** chapter 2 answers “what does skew look like”, not “what do I do about it”.

2 The evidence this run produced

Where	Number	Reading
key distribution	`C0YDPQWPBJ` = 17,539,732 rows; next key = 7,999	44 % of the table in one key; key ratio 2,193×
stage 27, duration	min 20 ms · median 33 ms · **max 4 s**	skew ratio 134× — anything over 5× is suspicious
stage 27, shuffle read	median 110,390 recs · **max 17,684,814 recs**	160×; that task alone read 3.5 MiB of the 8.1 MiB
stage 27, spill	**416 MiB memory / 2.8 MiB disk**, median 0 B	the fat task ran out of execution memory and wrote to disk
stage 27, wall clock	duration 5 s, total task time 13 s on 4 cores	perfectly spread this would be ~3.3 s — the tail costs ~40 %
whole join	`count()` = 39,790,092 in 7.82 s	199 of 200 tasks were done after ~0.6 s

> **Gotcha:** the Jobs page looks healthy — 214/214 tasks green, 8 s. Skew never shows in the job summary; you have to open the stage.

> **One-liner:** a stage costs what its slowest task costs, so compare Max against Median, never against the average.

3 The plan of the skewed join (the “before”)

`df_transactions.join(df_customers, on="cust_id", how="inner")` with broadcast disabled. Attribute ids, `plan_id` and `ENSURE_REQUIREMENTS` stripped:

```
*(5) Project [cust_id, start_date, end_date, txn_id, date, year, month, day, expense_type, amt, city, name, ...]
+- *(5) SortMergeJoin [cust_id], [cust_id], Inner
   :- *(2) Sort [cust_id ASC NULLS FIRST], false, 0
   :  +- Exchange hashpartitioning(cust_id, 200)       ← skew is decided HERE
   :     +- *(1) Filter isnotnull(cust_id)
   :        +- FileScan parquet transactions  [163 files, 39,790,092 rows]
   +- *(4) Sort [cust_id ASC NULLS FIRST], false, 0
      +- Exchange hashpartitioning(cust_id, 200)
         +- *(3) Filter isnotnull(cust_id)
            +- FileScan parquet customers     [1 file, 5,000 rows]
```

Read it bottom-up: scan → filter → **hash-partition both sides on `cust_id` into 200** → sort each partition → merge. The hash function is deterministic, so every row of `C0YDPQWPBJ` is sent to the same partition — partition 52 in this run. No amount of memory or cores changes that; only changing the *key* or the *join strategy* does.

> **One-liner:** find the `Exchange hashpartitioning(key, n)` line — that is the line that creates the skew, and the line every fix has to change.

4 ★ The fixes, measured on this same data

Same session, same 39.79 M ⋈ 5,000 join, same `local[4]` box. Timings are one run each — expect ±15 % between runs.

Option	Switch / code	Plan marker it produces	Time
A — baseline (the notebook)	`adaptive=false`, `autoBroadcast=-1`	`SortMergeJoin` + 2 × `Exchange`	9.40 s
B — AQE on, defaults	`spark.sql.adaptive.enabled=true`	`AQEShuffleRead coalesced` — **no skew split**	7.18 s
C — AQE + skew thresholds lowered	+ `skewJoin.skewedPartitionThresholdInBytes=256k`, `skewedPartitionFactor=2`, `advisoryPartitionSizeInBytes=64k`	**`SortMergeJoin(skew=true)`**, `AQEShuffleRead coalesced and skewed`	4.70 s
D — broadcast the small side	`autoBroadcastJoinThreshold=10MB` (the default) or `F.broadcast(df_customers)`	**`BroadcastHashJoin`**, **no Exchange at all**	**1.16 s**
E — salting, N = 16	salt big side with `rand()*N`, `explode` the small side, join on (key, salt)	`SortMergeJoin [cust_id, salt]` + `Generate explode`	6.53 s

> **The result that matters:** option B did nothing about the skew. AQE only splits a partition when it is bigger than `spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes` (**256 MB** by default) *and* more than `skewedPartitionFactor` (5×) the median. Here the fat partition is 17.5 M rows but only ~3.5 MiB of shuffle bytes, because one repeated 10-character string compresses to almost nothing. Below 256 MB, AQE shrugs. **Having AQE on is not the same as having skew handled.**

> **One-liner:** on this dataset the fix is broadcast (8× faster) — salting is the answer only when both sides are large.

5 The same query after each fix (plans side by side)

B — AQE on, default thresholds: coalesced only

```
AdaptiveSparkPlan isFinalPlan=true
+- == Final Plan ==
   ...
   +- *(5) SortMergeJoin [cust_id], [cust_id], Inner        ← still a plain sort-merge join
      :- *(3) Sort [cust_id ASC NULLS FIRST], false, 0
      :  +- AQEShuffleRead coalesced                        ← small partitions merged, fat one untouched
      :     +- ShuffleQueryStage 0
      :        +- Exchange hashpartitioning(cust_id, 200)
```

C — AQE with the skew thresholds lowered: the partition is split

```
AdaptiveSparkPlan isFinalPlan=true
+- == Final Plan ==
   ...
   +- *(5) SortMergeJoin(skew=true) [cust_id], [cust_id], Inner
      :- *(3) Sort [cust_id ASC NULLS FIRST], false, 0
      :  +- AQEShuffleRead coalesced and skewed              ← fat partition cut into sub-partitions
      :     +- ShuffleQueryStage 0
      :        +- Exchange hashpartitioning(cust_id, 200)
      +- *(4) Sort [cust_id ASC NULLS FIRST], false, 0
         +- AQEShuffleRead coalesced                        ← matching rows of the small side replicated
```

D — broadcast: the shuffle disappears

```
*(2) Project [...]
+- *(2) BroadcastHashJoin [cust_id], [cust_id], Inner, BuildRight
   :- *(2) Filter isnotnull(cust_id)
   :  +- FileScan parquet transactions  [163 files]        ← big side never moves
   +- BroadcastExchange HashedRelationBroadcastMode(...)
      +- *(1) Filter isnotnull(cust_id)
         +- FileScan parquet customers  [1 file, 5,000 rows]
← note what is GONE: no Exchange hashpartitioning, no Sort, no 200 partitions
```

E — salting: the key itself changes

```
*(5) SortMergeJoin [cust_id, salt], [cust_id, salt], Inner     ← composite key spreads the hot key
   :- *(2) Sort [cust_id ASC, salt ASC], false, 0
   :  +- Exchange hashpartitioning(cust_id, salt, 200)
   :     +- *(1) Project [..., cast((rand() * 16.0) as int) AS salt]   ← big side: random salt 0..15
   +- *(4) Sort [cust_id ASC, salt ASC], false, 0
      +- Exchange hashpartitioning(cust_id, salt, 200)
         +- *(3) Generate explode([0,1,...,15])                    ← small side: 16 copies of every row
```

> **One-liner:** broadcast removes the Exchange, AQE re-reads it, salting re-keys it — three different answers to the same line of the plan.

6 The full menu of fixes (including the ones not in this series)

Fix	How	Use it when
Broadcast join	`F.broadcast(small_df)`, or leave `autoBroadcastJoinThreshold` at 10 MB and let Spark decide	one side fits in executor memory (≤ ~100 MB after decompression). **Always try this first.**
AQE skew join	`adaptive.enabled=true` + `adaptive.skewJoin.enabled=true`; tune `skewedPartitionThresholdInBytes` / `skewedPartitionFactor` if partitions are small in bytes	sort-merge join, both sides big, you do not want to change code. Free, but check the plan says `skew=true`.
Salting	big side: `withColumn("salt", (rand()*N).cast("int"))`; small side: `explode(array(0..N-1))`; join on (key, salt)	both sides large and AQE cannot help; you accept N× blow-up of the replicated side
Isolate the hot key	filter the one key out, join it separately (usually broadcast), `union` the results	one or two known hot keys — cheapest targeted fix, no N× blow-up
Pre-aggregate	aggregate before the join, or two-step aggregation (partial with salt → final without)	skew in a `groupBy` rather than a join
Drop the junk key	`where(key.isNotNull())` or filter the sentinel (`"UNKNOWN"`, `-1`, `""`)	the hot key is a null/placeholder you do not need — surprisingly often the real answer
More shuffle partitions	`spark.sql.shuffle.partitions`	never fixes a single hot key (hashing still sends it to one partition); it only helps when the skew is mild and spread
Bucketing	write both tables bucketed on the join key	repeated joins on the same key in a warehouse; removes the shuffle but not the per-key imbalance

Order of attack: 1) is a side small enough to broadcast? 2) can the hot key be dropped or isolated? 3) is AQE on *and* actually firing (`skew=true` in the plan)? 4) only then, salt.

7 Interview extras

Q. How do you know a Spark job is skewed?

> *"I open the stage, not the job. In Summary Metrics I compare Max against Median for duration and for shuffle-read records — on this run the median task was 33 ms and the max was 4 s, 134×, and the same task spilled 416 MiB. The event timeline confirms it: 199 of 200 tasks finished in under a second and the stage waited four more seconds for one bar."*

Q. Why does more parallelism not fix skew?

> *"Because the partitioning is by hash of the key. Every row with the same key goes to the same partition whatever* `spark.sql.shuffle.partitions` *is. 200 or 2,000, that one customer is still one task. You have to change the key (salting), the strategy (broadcast), or split the partition at runtime (AQE skew join)."*

Q. You have AQE on. Are you safe?

> *"Not automatically. AQE only splits a partition over 256 MB by default and at least 5× the median. In my run the hot partition was 17.5 M rows but only 3.5 MiB of shuffle bytes because the key is a short repeated string, so AQE coalesced and moved on — the plan said* `AQEShuffleRead coalesced` *and not* `skew=true`*. I lowered* `skewedPartitionThresholdInBytes` *and then it split and the join went from 7.2 s to 4.7 s."*

Q. What does skew actually cost?

> *"Three things: wall clock is set by one task while the other cores idle, so you pay for cluster time you are not using; that one task can spill or OOM because it holds the whole key in memory; and retries get expensive because the one fat task is the one most likely to fail."*

Study order: this chapter (see it) → chapter 3 AQE + broadcast (fix it cheaply) → chapter 4 salting (fix it when nothing else works). Before the interview, be able to draw the event timeline and say the two numbers: median task and max task.